

TexTune Batch Split Engine – Explanation

Frontend: <https://thermotune-task.vercel.app>

Backend: <https://textune-api.onrender.com>

Approach

The system implements one closed loop:

1. Order input
2. Generate valid batch plans
3. Rank and explain them
4. Planner accepts or edits
5. Validate and save the final decision
6. Measure the difference from the recommendation
7. Feed that decision into future rankings

Each stage is a small, independently testable unit rather than one large function, so the reasoning behind any single recommendation can be traced step by step:

1. **Generation** produces every practical way to split the order quantity into batches within the allowed range, using a backtracking search rather than brute-force enumeration. Candidate batch sizes come from the preferred sizes given in the request plus the minimum, the maximum, and a fixed step in between (default 50kg), so a valid split can usually be found even when the preferred sizes alone cannot reach the exact total. The search prunes a branch the moment the remaining quantity drops below the minimum batch size, since no legal batch can close it out from there. Each finished plan is canonicalized as a sorted-descending tuple before being recorded, so 800+1000+600 and 1000+600+800 collapse into the same suggestion instead of being treated as two. A node-count cap (5000) bounds the search so a wide min/max range with a fine step cannot blow up computation on a large order.
2. **Ranking** turns the pool of valid plans into one ordered list using a small set of named, independent scoring rules rather than one opaque formula: a bonus for matching a preferred size, a bonus for equal-sized batches, a penalty for a batch sitting near the minimum, a penalty for using more batches than the minimum possible count, and a capped bonus from historical data. Each rule contributes both a score delta and (when it fires) a short reason phrase, so the same computation that produces the ranking also produces the explanation, and the two can never contradict each other.
3. **Explanation** is built by joining the reason phrases that actually fired, worded differently depending on whether the plan is the top recommendation or an alternative, matching the tone of the assignment's own worked examples.

4. **Editing** happens in the frontend's batch editor (add, remove, split, merge, resize), with the same validation rules mirrored client-side for instant feedback and re-run server-side as the actual gate before a decision is ever saved.
5. **Feedback capture** classifies what changed (accepted, resized, split, merged, added, removed, replaced, or several of these together) by comparing the initial and final plans as multisets, and computes how similar the two plans are.
6. **Measurement and learning** roll completed decisions up into acceptance rate, average changes, and batch/pattern statistics, which feed back into the ranking bonus for future orders of a similar size.

Assumptions

- Batch quantities are whole kilograms; the API still accepts floats but the generator, validator, and UI are designed around integer-like values.
- Available batch sizes are preferences, not hard restrictions. The generator also derives candidates from the minimum, maximum, and a fixed step, so a valid plan is not limited to only the preferred sizes.
- All batches in a plan share the same minimum and maximum limits; there is no per-batch override.
- A final plan can never be saved unless its total exactly matches the order quantity within floating-point tolerance.
- Historical learning affects ranking only. It can nudge which plan is recommended, but it can never make an otherwise-invalid plan pass validation, and it can never be the reason a plan is rejected.
- User preference is inferred only from completed decisions (a saved UserDecision row), not from suggestions that were generated but never acted on.
- Recent decisions count more than old ones (a 30-day half-life), reflecting the assumption that recent planner behavior is more representative of current preference than behavior from months ago.
- The optional fields from the request schema (machine_capacity_kg, delivery_priority, product_type, planner_id) are accepted and stored on the request but are not yet used in ranking or historical learning; they are reserved for the product/machine-specific preferences listed under future improvements.

How Suggestion Quality is Measured

GET /api/analytics/summary reports:

- **Acceptance rate:** the percentage of completed decisions where the final plan exactly matched the recommendation, with no edits at all.

- **Average changes per order:** the mean number of batch "slots" that ended up a different size between the initial and final plan (not simply the number of change *types*, since a single decision can carry several types at once).
- **Most selected batch size** and **most common change reason**, both simple frequency counts over completed decisions.
- **Average similarity score**, the mean of the per-decision similarity described below.

Similarity between the initial and final plan is a blend of two signals, each capturing something the other misses on its own:

- **Quantity similarity:** sort both plans descending, pad the shorter one with zeros, sum the absolute difference at each position, and express that as a fraction of $2 \times \text{order quantity}$ (the factor of two accounts for the fact that moving quantity from one batch to another shows up as a difference on both sides). A plan identical to the recommendation scores 100%; a completely different one approaches 0%.
- **Structure similarity:** how much the batch *count* changed, independent of the quantities involved, since going from three batches to four is a meaningfully different plan even if the total quantity moved is small.

The two are combined as $0.8 \times \text{quantity_similarity} + 0.2 \times \text{structure_similarity}$, clamped to the 0-100% range. The 80/20 weighting reflects that quantity movement is the primary signal (it is what the assignment's own worked example is built around) while batch-count change is a secondary, structural correction on top of it.

How Historical Feedback Improves Future Suggestions

Historical learning is deliberately rule-based counting rather than a trained model, for reasons explained in AI_USAGE.md's "Important Decisions Made Independently" section: the dataset stays small, explainability matters for the interview walkthrough, and the assignment itself states a rule-based approach is sufficient at this scale. Two independent signals are computed from completed decisions and combined into one capped, additive bonus during ranking:

- **Batch acceptance rate:** for every batch size that has appeared in a *recommended* suggestion, what fraction of the time it also survived unchanged into the final plan. A batch size that keeps getting removed or resized away accumulates a low rate; one planners consistently keep accumulates a high one. Only batch sizes that were actually offered in a recommendation are tracked this way, so a value the planner introduced purely through editing does not get credit here (that is a known, documented limitation, not a bug).
- **Similar-order pattern matching:** for a new order, look up completed decisions on past orders within a configurable quantity range (default $\pm 10\%$) and reward a candidate plan that exactly matches a plan chosen before in that range.

Both signals are recency-weighted (a 30-day half-life, so a decision from a month ago counts about half as much as one from today) and both are capped before being added to the score, with the combined historical bonus capped again on top of that. This means historical data can shift which

plan looks best, and can visibly move a previously-buried plan up several ranks, but it can never be large enough to override the validation-scale penalties from the ranking rules above it, and it can never bypass the hard validation rules (sum, min, max) applied when a decision is actually saved.

Future Improvements

- Train a model on accumulated accepted/modified decisions once there is enough data to validate one meaningfully, rather than reaching for one now on a small dataset.
- Use the optional `machine_capacity_kg` and `product_type` fields to bias ranking toward machine- or product-specific historical patterns, instead of only order-quantity-based similarity.
- Planner-specific preferences using `planner_id`, rather than pooling all planners' decisions together.
- Tunable recency half-life exposed as configuration rather than a fixed constant.
- Exportable analytics reports and visual charts on the dashboard.
- Drag-and-drop batch editing as an alternative to the current form-based editor.

System Architecture Diagram

